

Spring Data JPA E-Commerce Backend

Database Integration with Spring Boot, Spring Data JPA and PostgreSQL

Project	Week 7 - Database Integration with Spring Data JPA
Version	v1.0.0
Author	Ashutosh Panwar
Language	Java 21
Framework	Spring Boot 4.0.8
Database	PostgreSQL (production), H2 (development/testing)
Build	Maven
Repository	github.com/Ashutosh9-pan/week7-spring-data-jpa
Documentation date	17 September 2026

Release: v1.0.0 - Latest

This document covers the implemented architecture, database design, REST API behavior, transaction strategy, performance configuration, testing, setup, and evidence screenshots for the submitted Week 7 project.

1. Documentation Contents

- 2. Project Overview and Objectives
- 3. Technology Stack
- 4. Architecture and Code Structure
- 5. Database Design and Relationships
- 6. Flyway Migrations
- 7. Repository and Query Design
- 8. Transactions, Locking and Order Processing
- 9. Auditing, Caching and Performance
- 10. REST API Summary
- 11. Testing and Verification
- 12. Setup and Installation
- 13. Requirement Coverage
- 14. Evidence Screenshots
- 15. Release and Repository
- 16. Future Enhancements

Note: The documentation describes the implemented project scope. The current database schema contains users, products, categories, orders, order_items and payments; there is no separate persistent cart table in the implemented schema.

2. Project Overview and Objectives

The Week 7 project is a layered RESTful e-commerce backend focused on relational database integration. It uses Spring Data JPA and Hibernate for persistence, PostgreSQL as the production database, H2 for development/testing, Flyway for schema versioning, and Spring-managed transactions for inventory-sensitive order processing.

Core objectives

- Integrate Spring Data JPA with a relational database and reusable repository interfaces.
- Use derived queries, custom JPQL queries, entity graphs and pagination.
- Implement transaction management and pessimistic locking for stock-sensitive order creation.
- Manage schema changes with Flyway migrations and validate the database schema at startup.
- Implement JPA auditing for createdAt and updatedAt timestamps.
- Use caching for frequently accessed product data.
- Configure HikariCP connection pooling and SQL/JDBC logging for performance visibility.
- Seed validated development data and verify behavior through automated repository and service tests.
- Expose interactive REST documentation through Swagger/OpenAPI.

Implemented business areas

User registration/login and profile management; product CRUD, search, filtering, pagination and sorting; hierarchical categories; order creation, status updates and cancellation; payment creation/status management; and daily order reporting.

3. Technology Stack

Technology	Role
Java 21	Programming language
Spring Boot 4.0.8	Application framework
Spring Data JPA	Repository and persistence abstraction
Hibernate	JPA/ORM implementation
Spring Web MVC	REST API layer
PostgreSQL	Production relational database
H2	Development and test database
Flyway	Versioned database migrations
HikariCP	JDBC connection pooling
Spring Cache	Product retrieval caching
BCrypt	Password hashing
Jakarta Validation	Request/entity validation
Springdoc OpenAPI	Swagger/OpenAPI documentation
Maven	Build and dependency management
JUnit / Spring Test	Automated tests

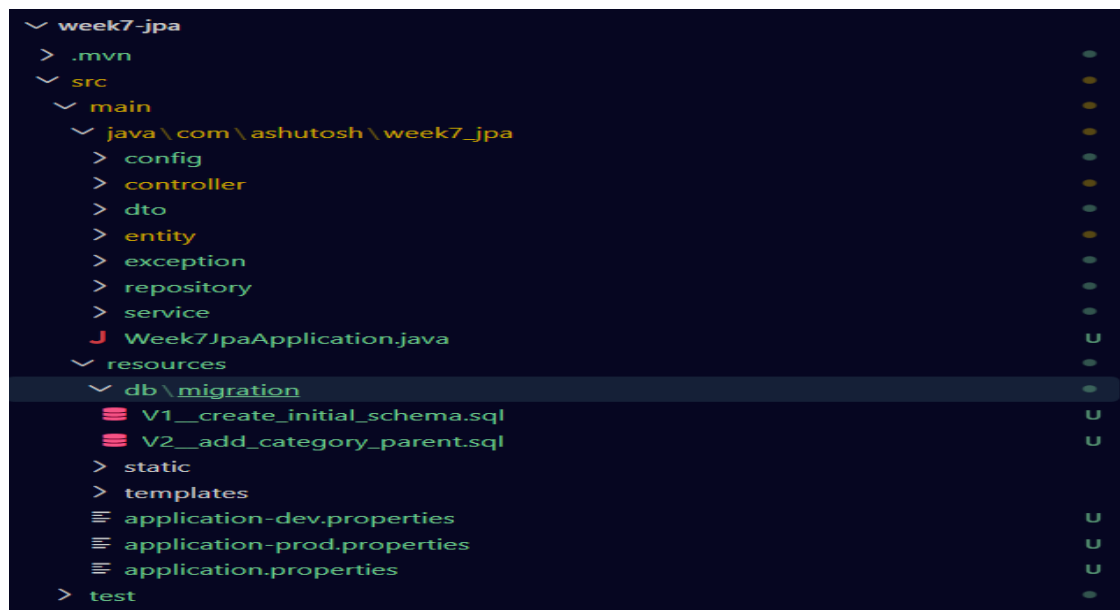
4. Architecture and Code Structure

The application follows a layered Spring architecture:

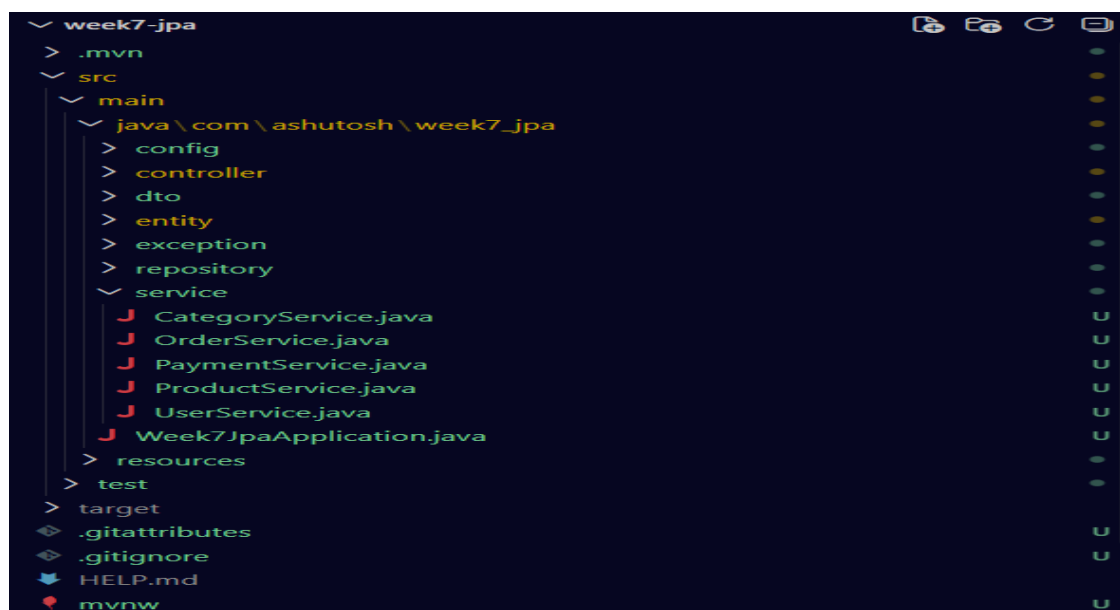
```
com.ashutosh.week7_jpa
config - caching, auditing, seeding, security and OpenAPI configuration
controller - REST endpoints
dto - request/response DTOs
entity - JPA entities and enums
exception - domain/resource error handling
repository - Spring Data JPA repositories and custom queries
service - business logic and transaction boundaries
resources
db/migration - Flyway V1 and V2 migrations
application*.properties - environment/profile configuration
test - repository, service, caching, auditing and seeding tests
```

Evidence

The screenshots below show the project package layout, service/controller organization, and Maven/runtime configuration.



Project package tree

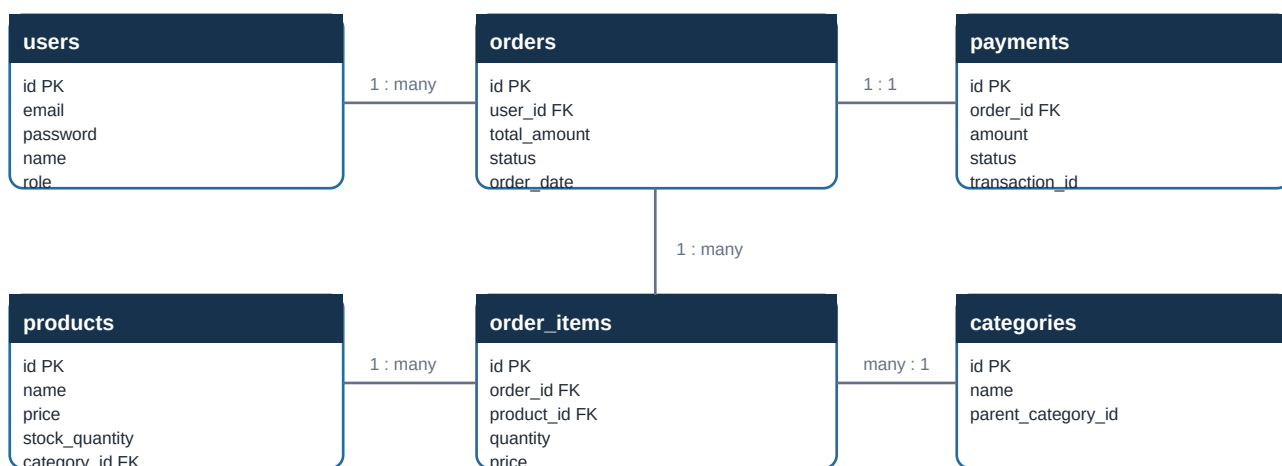


Service layer package tree

5. Database Design and Relationships

Primary tables and relationships:

Table	Key role
users	User identity, email, hashed password, role and audit timestamps
categories	Product categories plus parent_category_id hierarchy
products	Product catalog, pricing, stock, active flag and category relationship
orders	Order header, user relationship, status, total amount and order date
order_items	Order-to-product line items with quantity and price
payments	Payment record associated with an order, amount, method/status and transaction id



Conceptual relationship view based on the implemented JPA mappings and schema.

Category hierarchy

Categories support a self-referencing parent_category_id relationship. This allows root categories and child categories such as Electronics -> Laptops.

Indexes and fetch optimization

The implementation defines database indexes for commonly filtered fields and uses LAZY relationships plus EntityGraph-based fetches where related data is needed together.

6. Flyway Migrations

Flyway keeps the relational schema versioned and repeatable across environments.

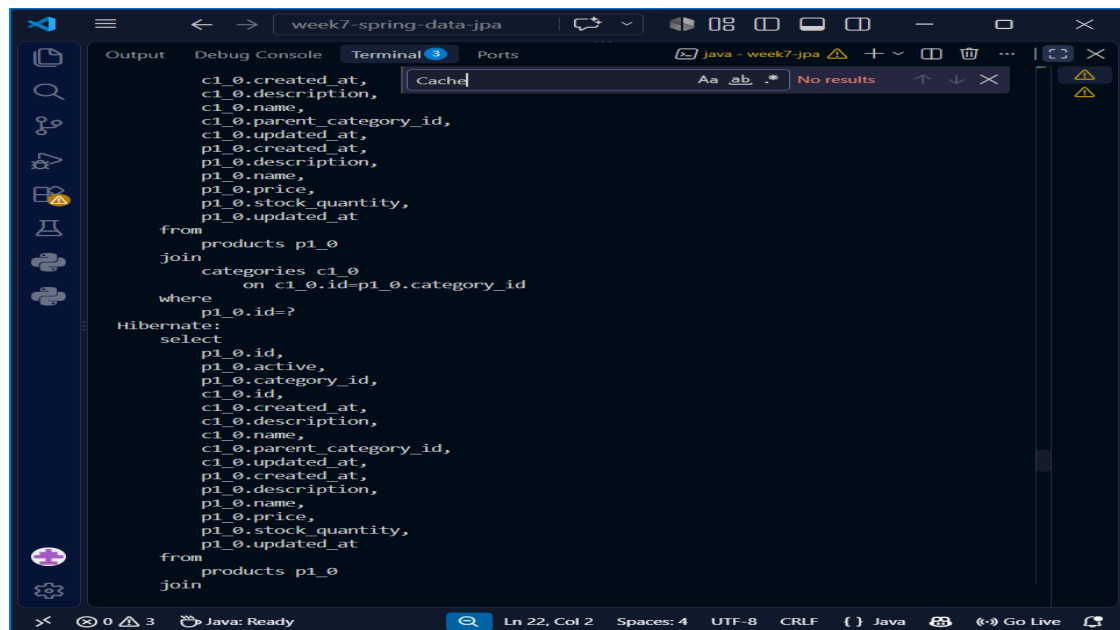
Migration	Purpose
V1__create_initial_schema.sql	Creates the initial users, categories, products, orders, order_items and payments schema, constraints and indexes.
V2__add_category_parent.sql	Adds categories.parent_category_id, the self-referencing foreign key and parent-category index.

The development profile uses an in-memory H2 database in PostgreSQL compatibility mode, so a clean application restart can re-apply migrations. Production uses PostgreSQL with schema validation and Flyway enabled.

7. Repository and Query Design

- Spring Data JPA repositories provide derived queries for common filters such as active products, name matching, category filtering and user lookup.
- Custom JPQL queries are used where the query needs combined optional filters, explicit ordering or aggregate reporting.
- EntityGraph is used to fetch related category/order/payment data efficiently for selected read operations.
- Pagination uses Spring Data Page/Pageable to avoid loading an unbounded product result set.
- Pessimistic write locking is used by the product repository for stock-sensitive order creation.

Evidence: Hibernate query logging



```

c1_0.created_at,
c1_0.description,
c1_0.name,
c1_0.parent_category_id,
c1_0.updated_at,
p1_0.created_at,
p1_0.description,
p1_0.name,
p1_0.price,
p1_0.stock_quantity,
p1_0.updated_at
from
products p1_0
join
categories c1_0
on c1_0.id=p1_0.category_id
where
p1_0.id=?
Hibernate:
select
p1_0.id,
p1_0.active,
p1_0.category_id,
c1_0.id,
c1_0.created_at,
c1_0.description,
c1_0.name,
c1_0.parent_category_id,
c1_0.updated_at,
p1_0.created_at,
p1_0.description,
p1_0.name,
p1_0.price,
p1_0.stock_quantity,
p1_0.updated_at
from
products p1_0
join

```

Runtime SQL evidence showing a product/category join generated by Hibernate.

8. Transactions, Locking and Order Processing

Order creation is handled as a transactional business operation. The service validates the user and requested products, locks products for update, checks stock, deducts inventory, calculates the order total and persists the order. If an error occurs inside the transaction, the operation can roll back instead of leaving partial inventory/order state.

Step	Operation
1	Validate active user
2	Resolve requested products
3	Acquire pessimistic write locks
4	Validate available stock
5	Deduct stock
6	Calculate line totals and order total
7	Persist order and order items
8	Commit transaction

Cancellation reverses the stock deduction for the order items and marks the order as CANCELLED within the service transaction.

9. Auditing, Caching and Performance

JPA auditing

A reusable BaseAuditableEntity uses @CreateDate and @LastModifiedDate. JpaAuditConfig enables Spring Data JPA auditing so timestamps are maintained automatically.

Product caching

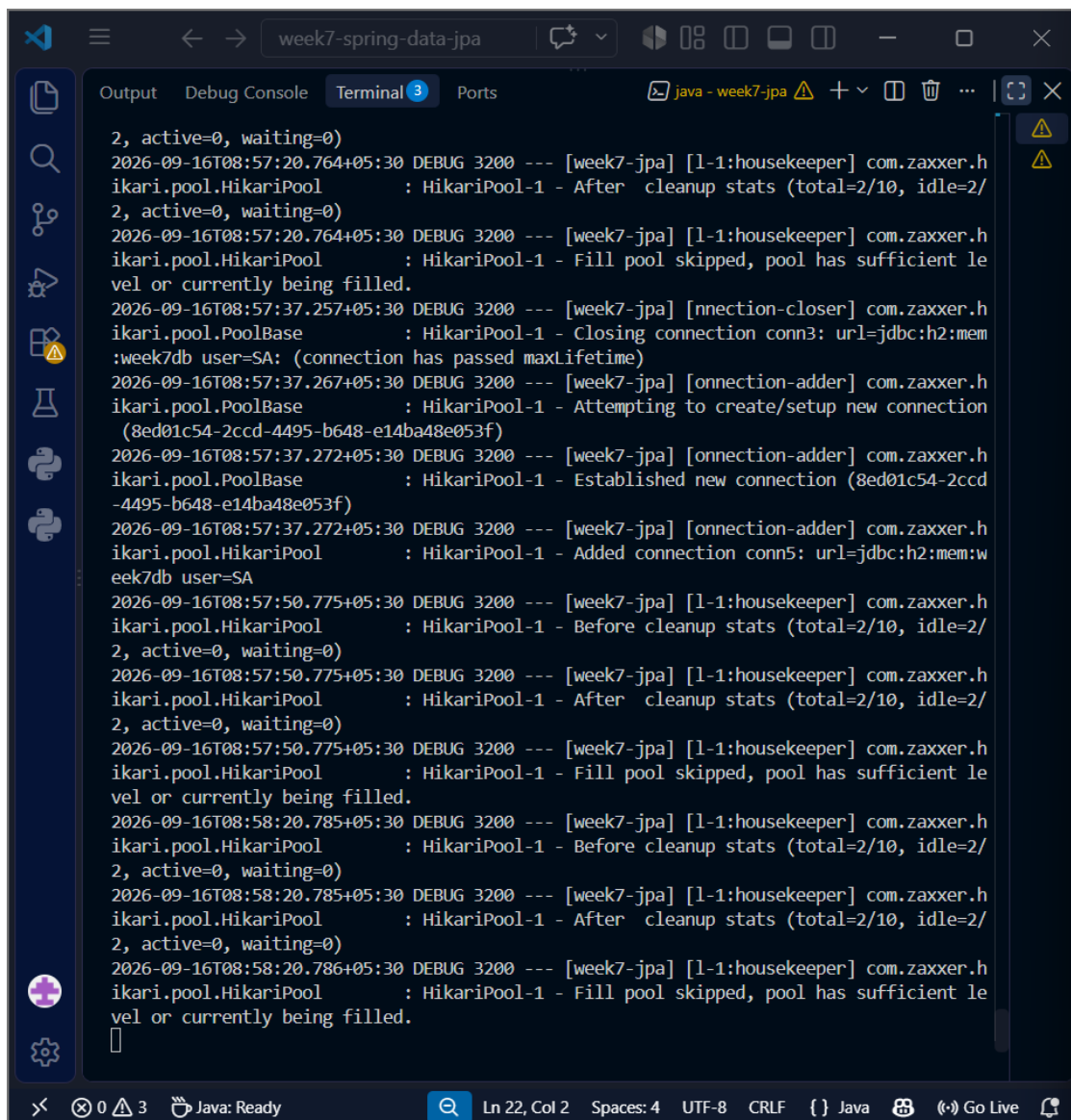
Product retrieval uses Spring Cache annotations: @Cacheable for reads, @CachePut for create/update and @CacheEvict for deletion.

HikariCP

The application configures maximum pool size, minimum idle connections, connection timeout, idle timeout and maximum lifetime. Runtime logs show HikariPool activity.

Query logging

Hibernate SQL and JDBC bind logging are enabled in development to make query behavior visible during integration testing and debugging.



The screenshot shows a terminal window with the title "week7-spring-data-jpa". The terminal displays a series of debug logs from HikariCP. The logs show the pool's state (total=2/10, idle=2/2, active=0, waiting=0) and various actions such as cleanup, connection skipping, and connection establishment. The logs are timestamped and include the thread name [1-1:housekeeper] and the class com.zaxxer.hikari.pool.HikariPool. The IDE interface includes a sidebar with icons for Explorer, Search, Run and Debug, and Extensions. The bottom status bar shows "Ln 22, Col 2" and "Spaces: 4 UTF-8 CRLF {} Java".

```
2, active=0, waiting=0)
2026-09-16T08:57:20.764+05:30 DEBUG 3200 --- [week7-jpa] [1-1:housekeeper] com.zaxxer.h
ikari.pool.HikariPool : HikariPool-1 - After cleanup stats (total=2/10, idle=2/
2, active=0, waiting=0)
2026-09-16T08:57:20.764+05:30 DEBUG 3200 --- [week7-jpa] [1-1:housekeeper] com.zaxxer.h
ikari.pool.HikariPool : HikariPool-1 - Fill pool skipped, pool has sufficient le
vel or currently being filled.
2026-09-16T08:57:37.257+05:30 DEBUG 3200 --- [week7-jpa] [nnection-closer] com.zaxxer.h
ikari.pool.PoolBase : HikariPool-1 - Closing connection conn3: url=jdbc:h2:mem
:week7db user=SA: (connection has passed maxLifetime)
2026-09-16T08:57:37.267+05:30 DEBUG 3200 --- [week7-jpa] [onnection-adder] com.zaxxer.h
ikari.pool.PoolBase : HikariPool-1 - Attempting to create/setup new connection
(8ed01c54-2ccd-4495-b648-e14ba48e053f)
2026-09-16T08:57:37.272+05:30 DEBUG 3200 --- [week7-jpa] [onnection-adder] com.zaxxer.h
ikari.pool.PoolBase : HikariPool-1 - Established new connection (8ed01c54-2ccd
-4495-b648-e14ba48e053f)
2026-09-16T08:57:37.272+05:30 DEBUG 3200 --- [week7-jpa] [onnection-adder] com.zaxxer.h
ikari.pool.HikariPool : HikariPool-1 - Added connection conn5: url=jdbc:h2:mem:w
eek7db user=SA
2026-09-16T08:57:50.775+05:30 DEBUG 3200 --- [week7-jpa] [1-1:housekeeper] com.zaxxer.h
ikari.pool.HikariPool : HikariPool-1 - Before cleanup stats (total=2/10, idle=2/
2, active=0, waiting=0)
2026-09-16T08:57:50.775+05:30 DEBUG 3200 --- [week7-jpa] [1-1:housekeeper] com.zaxxer.h
ikari.pool.HikariPool : HikariPool-1 - After cleanup stats (total=2/10, idle=2/
2, active=0, waiting=0)
2026-09-16T08:57:50.775+05:30 DEBUG 3200 --- [week7-jpa] [1-1:housekeeper] com.zaxxer.h
ikari.pool.HikariPool : HikariPool-1 - Fill pool skipped, pool has sufficient le
vel or currently being filled.
2026-09-16T08:58:20.785+05:30 DEBUG 3200 --- [week7-jpa] [1-1:housekeeper] com.zaxxer.h
ikari.pool.HikariPool : HikariPool-1 - Before cleanup stats (total=2/10, idle=2/
2, active=0, waiting=0)
2026-09-16T08:58:20.785+05:30 DEBUG 3200 --- [week7-jpa] [1-1:housekeeper] com.zaxxer.h
ikari.pool.HikariPool : HikariPool-1 - After cleanup stats (total=2/10, idle=2/
2, active=0, waiting=0)
2026-09-16T08:58:20.786+05:30 DEBUG 3200 --- [week7-jpa] [1-1:housekeeper] com.zaxxer.h
ikari.pool.HikariPool : HikariPool-1 - Fill pool skipped, pool has sufficient le
vel or currently being filled.
```

HikariCP runtime logging evidence from the development environment.

10. REST API Summary

Area	Method	Endpoint	Purpose
Auth	POST	/api/auth/register	Register user
Auth	POST	/api/auth/login	Authenticate user
Users	GET	/api/users/profile	Get profile
Products	GET	/api/products	List/search/paginate
Products	GET	/api/products/{id}	Get product
Products	POST/PUT/DELETE	/api/products/{id}	Product management
Categories	GET	/api/categories*	Category hierarchy/read APIs
Categories	POST/PUT/DELETE	/api/categories/{id}	Category management
Orders	POST	/api/orders	Create order
Orders	GET	/api/orders/{id}	Read orders
Orders	PUT	/api/orders/{id}/status	Update order status
Orders	POST	/api/orders/{id}/cancel	Cancel order
Orders	GET	/api/orders/report/daily	Daily report
Payments	POST	/api/payments	Create payment
Payments	GET	/api/payments/{id}	Read payment

11. Testing and Verification

The project includes repository, service, caching, auditing, seeding and application-context tests. The final clean test run completed successfully with 24 tests, zero failures and zero errors.

Test area	Coverage
Application context	Spring Boot application startup wiring
Product repository	Derived/custom queries and pagination
Order service	Order creation, inventory and cancellation flow
Payment service	Payment creation and status behavior
Daily report	Order count and revenue aggregation
Caching	Product cache read/update/evict behavior
User service	Registration, hashing, login, inactive/unknown cases
Data seeding	Validated seed data and idempotence
JPA auditing	createdAt / updatedAt behavior

Final result: Tests run: 24 | Failures: 0 | Errors: 0 | Skipped: 0 | BUILD SUCCESS

12. Setup and Installation

Development setup

```
git clone https://github.com/Ashutosh9-pan/week7-spring-data-jpa.git
cd week7-spring-data-jpa
.\mvnw.cmd clean test
.\mvnw.cmd spring-boot:run
```

The default application configuration activates the dev profile. Development uses H2 in-memory PostgreSQL compatibility mode. When the application is running, Swagger UI is available at <http://localhost:8080/swagger-ui/index.html> and the H2 console at <http://localhost:8080/h2-console>.

Production configuration

The prod profile expects database connection environment variables (DB_URL, DB_USERNAME and DB_PASSWORD). Secrets should be supplied through the deployment environment rather than committed to source control.

13. Requirement Coverage

Requirement	Implementation evidence
PostgreSQL database / schema	PostgreSQL runtime profile plus V1/V2 Flyway schema migrations
Complex JPA relationships	User-Order, Order-OrderItem, OrderItem-Product, Product-Category, Order-Payment and self-referencing Category
Spring Data JPA repositories	JpaRepository interfaces with derived and custom JPQL queries
Custom queries	Product search/filtering, order detail fetches and daily aggregation
Transaction management	Transactional order creation/cancellation with stock consistency
Pessimistic locking	Product-for-update repository method used for inventory-sensitive order creation
Flyway	V1 initial schema and V2 category hierarchy migration
JPA auditing	BaseAuditableEntity + @CreatedDate/@LastModifiedDate
Indexes/fetch strategies	JPA @Index definitions and EntityGraph-based fetches
HikariCP	Configured pool limits/timeouts and runtime pool logging
Database seeding	Development seed configuration with validation/idempotence tests
Performance monitoring	Hibernate SQL/JDBC bind logging and HikariCP logs
API documentation	Springdoc OpenAPI / Swagger UI
Automated testing	24 tests, 0 failures, 0 errors

Scope note: the current implementation does not contain a separate persistent shopping-cart table. The documented order workflow covers the implemented purchase/order path.

14. Evidence Screenshots

The following evidence images were captured during development and API verification. They demonstrate the project structure, configuration, runtime behavior and working Swagger endpoints.

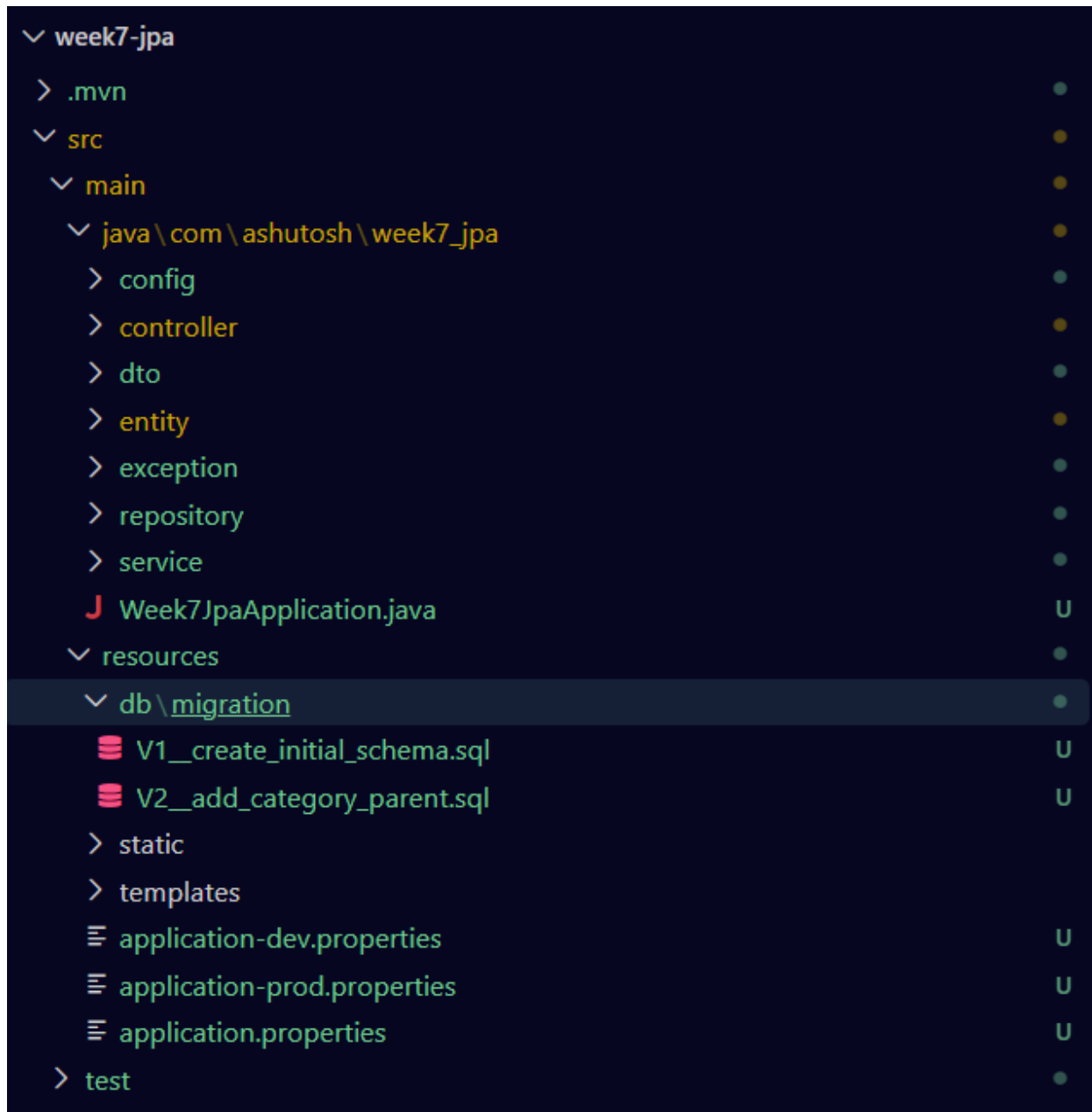


Figure 1. Project structure showing config, controller, DTO, entity, exception, repository, service and resource packages.



Figure 2. Service package organization, including *CategoryService*, *OrderService*, *PaymentService* and *ProductService*.



Figure 3. Controller package organization, including category, order, payment, product and user controllers.

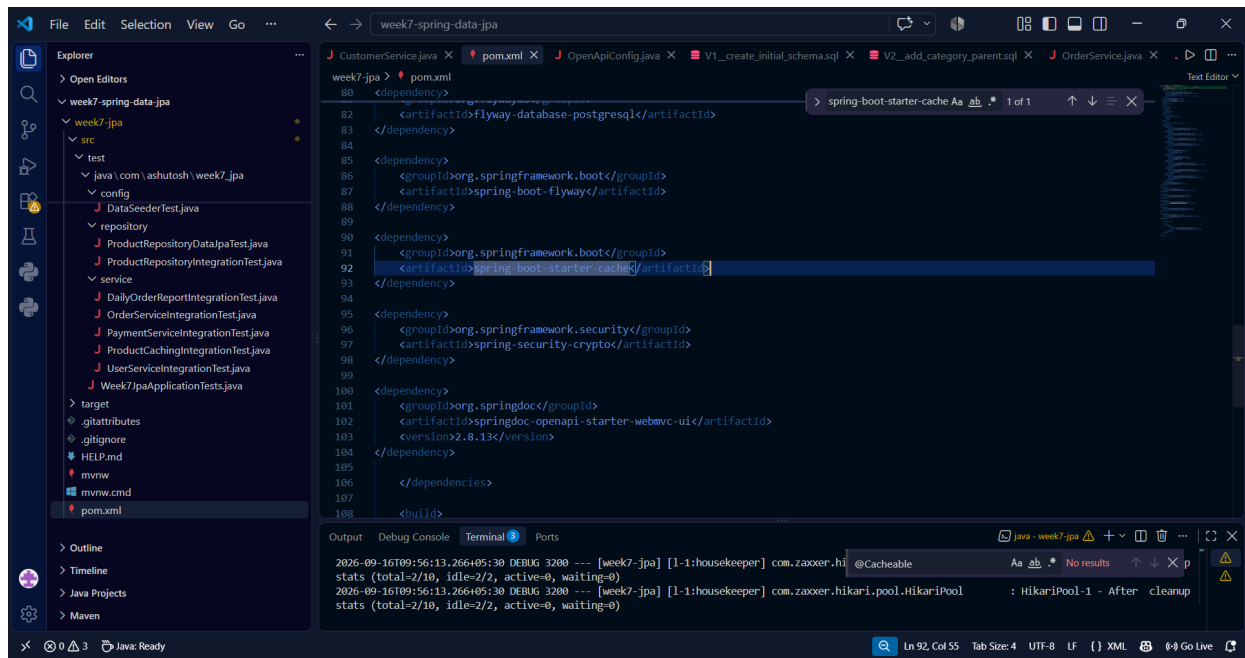


Figure 4. Maven dependency configuration for caching, Flyway, security crypto and OpenAPI.

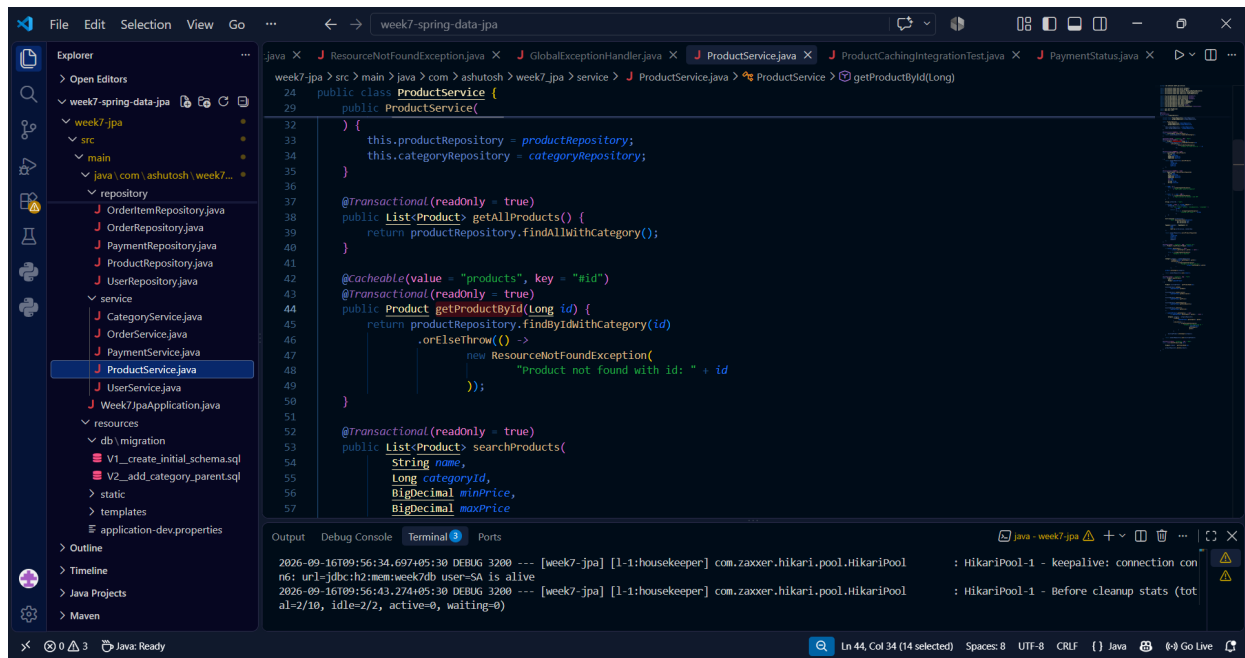
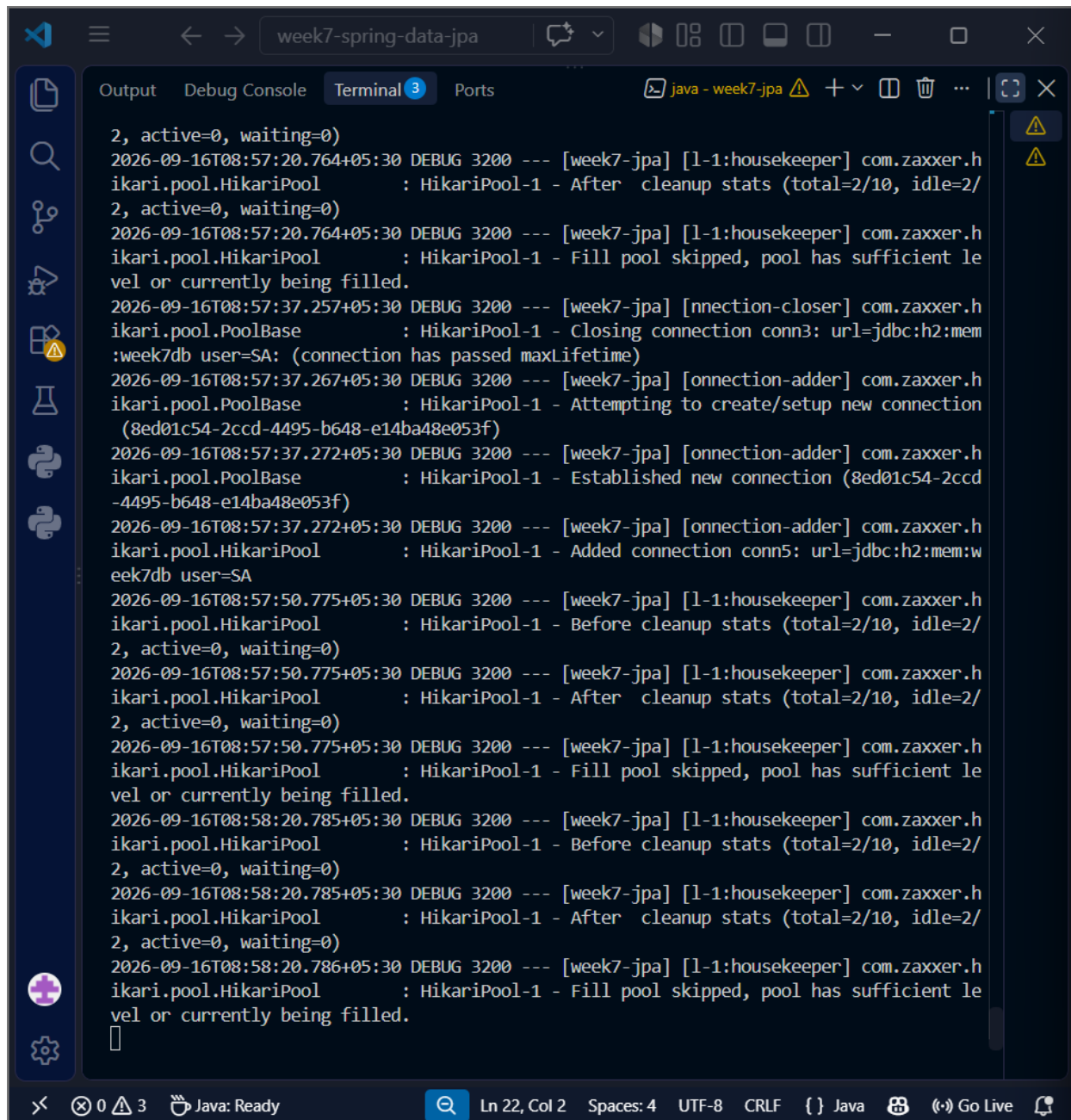


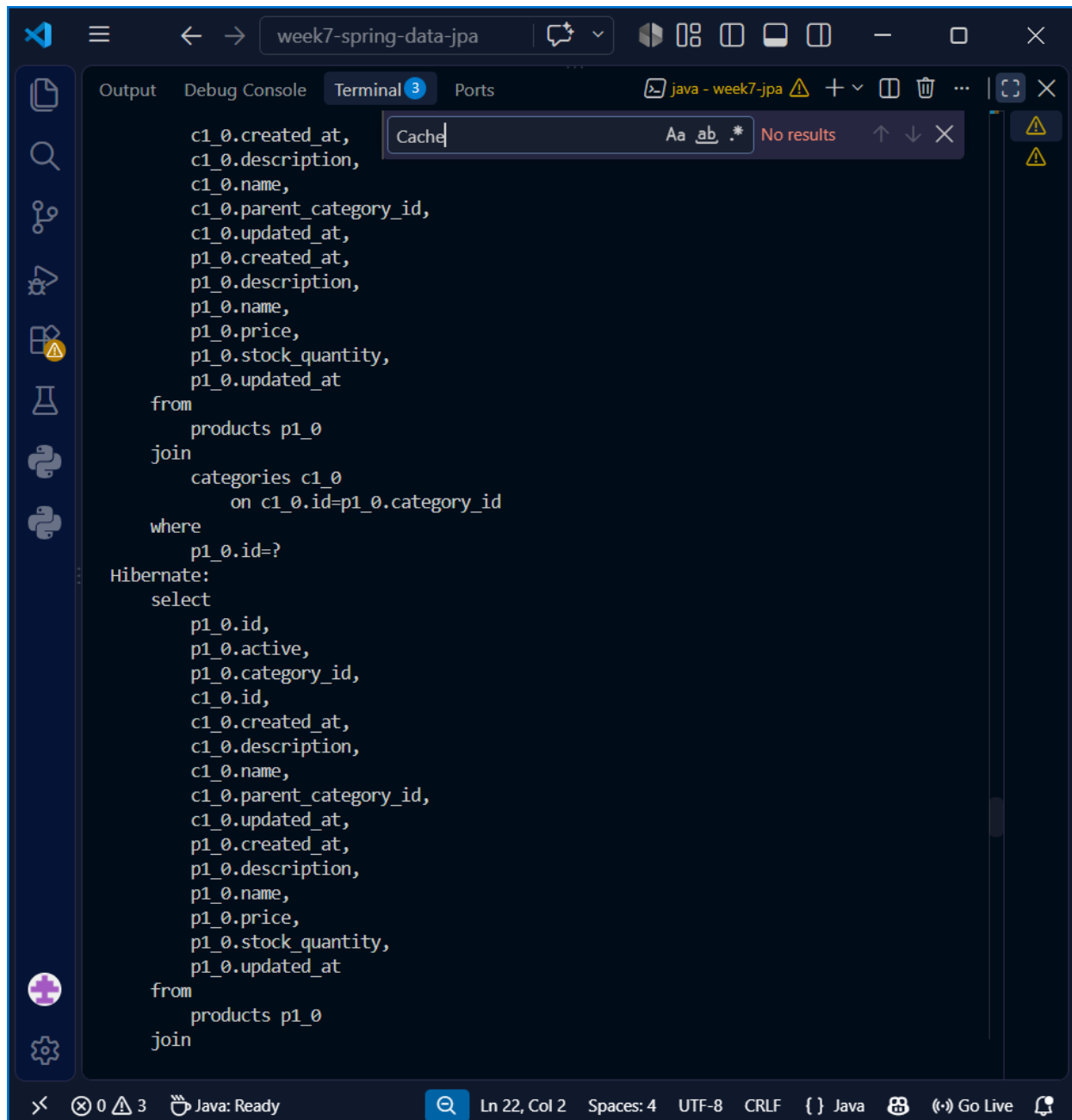
Figure 5. Integration-test source structure and order-service test implementation.



```
2, active=0, waiting=0)
2026-09-16T08:57:20.764+05:30 DEBUG 3200 --- [week7-jpa] [l-1:housekeeper] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - After cleanup stats (total=2/10, idle=2/2, active=0, waiting=0)
2026-09-16T08:57:20.764+05:30 DEBUG 3200 --- [week7-jpa] [l-1:housekeeper] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Fill pool skipped, pool has sufficient level or currently being filled.
2026-09-16T08:57:37.257+05:30 DEBUG 3200 --- [week7-jpa] [nnection-closer] com.zaxxer.hikari.pool.PoolBase : HikariPool-1 - Closing connection conn3: url=jdbc:h2:mem:week7db user=SA: (connection has passed maxLifetime)
2026-09-16T08:57:37.267+05:30 DEBUG 3200 --- [week7-jpa] [onnection-adder] com.zaxxer.hikari.pool.PoolBase : HikariPool-1 - Attempting to create/setup new connection (8ed01c54-2ccd-4495-b648-e14ba48e053f)
2026-09-16T08:57:37.272+05:30 DEBUG 3200 --- [week7-jpa] [onnection-adder] com.zaxxer.hikari.pool.PoolBase : HikariPool-1 - Established new connection (8ed01c54-2ccd-4495-b648-e14ba48e053f)
2026-09-16T08:57:37.272+05:30 DEBUG 3200 --- [week7-jpa] [onnection-adder] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added connection conn5: url=jdbc:h2:mem:week7db user=SA
2026-09-16T08:57:50.775+05:30 DEBUG 3200 --- [week7-jpa] [l-1:housekeeper] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Before cleanup stats (total=2/10, idle=2/2, active=0, waiting=0)
2026-09-16T08:57:50.775+05:30 DEBUG 3200 --- [week7-jpa] [l-1:housekeeper] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - After cleanup stats (total=2/10, idle=2/2, active=0, waiting=0)
2026-09-16T08:57:50.775+05:30 DEBUG 3200 --- [week7-jpa] [l-1:housekeeper] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Fill pool skipped, pool has sufficient level or currently being filled.
2026-09-16T08:58:20.785+05:30 DEBUG 3200 --- [week7-jpa] [l-1:housekeeper] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Before cleanup stats (total=2/10, idle=2/2, active=0, waiting=0)
2026-09-16T08:58:20.785+05:30 DEBUG 3200 --- [week7-jpa] [l-1:housekeeper] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - After cleanup stats (total=2/10, idle=2/2, active=0, waiting=0)
2026-09-16T08:58:20.786+05:30 DEBUG 3200 --- [week7-jpa] [l-1:housekeeper] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Fill pool skipped, pool has sufficient level or currently being filled.

```

Figure 6. HikariCP runtime logs showing pool creation and maintenance.



The screenshot shows an IDE window titled "week7-spring-data-jpa". The "Terminal" tab is active, displaying a Hibernate SQL query. A search bar at the top of the terminal shows "Cache" with "No results". The query is as follows:

```
c1_0.created_at,  
c1_0.description,  
c1_0.name,  
c1_0.parent_category_id,  
c1_0.updated_at,  
p1_0.created_at,  
p1_0.description,  
p1_0.name,  
p1_0.price,  
p1_0.stock_quantity,  
p1_0.updated_at  
from  
  products p1_0  
join  
  categories c1_0  
    on c1_0.id=p1_0.category_id  
where  
  p1_0.id=?  
Hibernate:  
select  
  p1_0.id,  
  p1_0.active,  
  p1_0.category_id,  
  c1_0.id,  
  c1_0.created_at,  
  c1_0.description,  
  c1_0.name,  
  c1_0.parent_category_id,  
  c1_0.updated_at,  
  p1_0.created_at,  
  p1_0.description,  
  p1_0.name,  
  p1_0.price,  
  p1_0.stock_quantity,  
  p1_0.updated_at  
from  
  products p1_0  
join
```

The IDE interface includes a sidebar with icons for Explorer, Search, Run and Debug, Extensions, Test Explorer, and Settings. The status bar at the bottom indicates "Java: Ready", "Ln 22, Col 2", "Spaces: 4", "UTF-8", "CRLF", and "Go Live".

Figure 7. Hibernate SQL logging showing a product/category join generated by JPA.

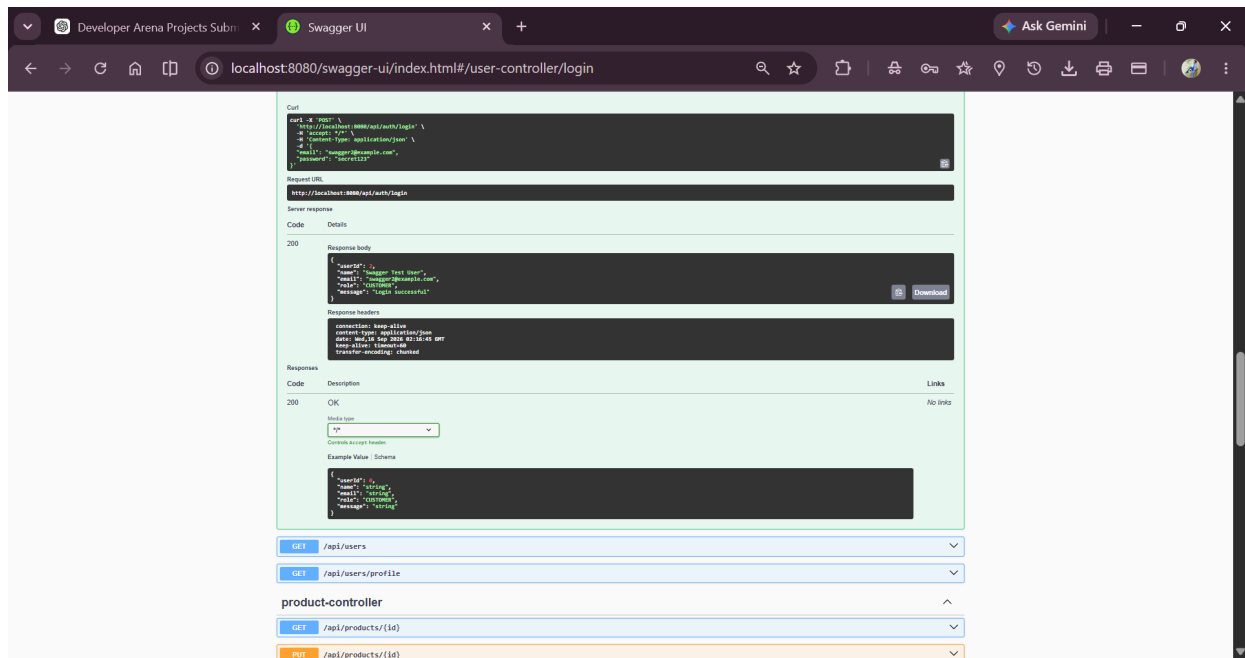


Figure 9. Swagger UI user login endpoint.

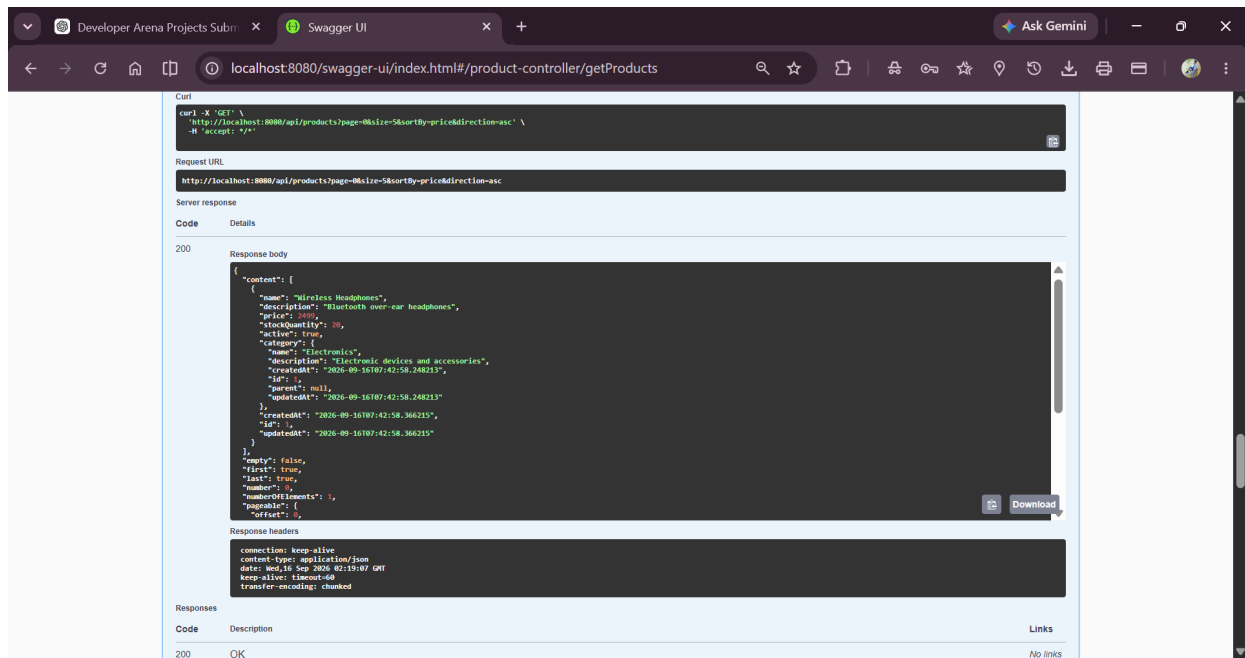


Figure 10. Swagger UI product listing endpoint with pagination parameters.

Developer Arena Projects Subn x Swagger UI x Ashutosh9-pan/week6-spring-1 x + Ask Gemini

localhost:8080/swagger-ui/index.html#/product-controller/getProductById

```
curl -X 'GET' \
  'http://localhost:8080/api/products/1' \
  -H 'accept: */*'
```

Request URL

http://localhost:8080/api/products/1

Server response

Code Details

200

Response body

```
{
  "name": "Wireless Headphones",
  "description": "Bluetooth over-ear headphones",
  "price": 2499,
  "stockQuantity": 20,
  "active": true,
  "category": {
    "name": "Electronics",
    "description": "Electronic devices and accessories",
    "createdAt": "2026-09-16T08:09:27.86758",
    "id": 1,
    "parent": null,
    "updatedAt": "2026-09-16T08:09:27.86758"
  },
  "createdAt": "2026-09-16T08:09:27.96105",
  "id": 1,
  "updatedAt": "2026-09-16T08:51:33.272942"
}
```

Response headers

```
connection: keep-alive
content-type: application/json
date: Wed, 16 Sep 2026 03:25:11 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	OK	No links

Media type: */*

Controls Accept header

Example Value Schema

Figure 11. Swagger UI product-by-ID endpoint.

Developer Arena Projects Subn x Swagger UI x Ashutosh9-pan/week6-spring-1 x + Ask Gemini

localhost:8080/swagger-ui/index.html#/payment-controller/updatePaymentStatus

```
curl -X 'PUT' \
  'http://localhost:8080/api/payments/1/status?status=COMPLETED' \
  -H 'Accept: */*'
```

Request URL

http://localhost:8080/api/payments/1/status?status=COMPLETED

Server response

Code Details

200

Response body

```
{
  "amount": 200,
  "paymentMethod": "UPI",
  "createdAt": "2026-09-16T08:36:15.049091",
  "id": 1,
  "paymentDate": "2026-09-16T08:36:15.065599",
  "status": "COMPLETED",
  "transactionId": "SWAGGER-10X-001",
  "updatedAt": "2026-09-16T08:43:46.288227"
}
```

Response headers

```
connection: keep-alive
content-length: 228
content-type: application/json
date: Wed, 16 Sep 2026 03:13:46 GMT
keep-alive: timeout=60
```

Responses

Code	Description	Links
200	OK	No links

Media type

/

Controls accept header

Example Value Schema

```
{
  "createdAt": "2026-09-16T03:13:46.3642",
  "updatedAt": "2026-09-16T03:13:46.3642",
  "id": 0,
  "amount": 0,
  "status": "PENDING",
  "paymentMethod": "string",
  "transactionId": "string",
  "paymentDate": "2026-09-16T03:13:46.3642"
}
```

Figure 12. Swagger UI payment status update endpoint.

Developer Arena Projects Subn... Swagger UI Ashutosh9-pan/CodeAlpha_Fl... Ask Gemini

localhost:8080/swagger-ui/index.html#/order-controller/getDailyOrderReport

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/api/orders/report/daily' \
  -H 'accept: */*'
```

Request URL

http://localhost:8080/api/orders/report/daily

Server response

Code	Details
200	Response body <pre>{ "date": "2025-09-15", "totalOrders": 0, "totalRevenue": 0 }</pre> Response headers <pre>connection: keep-alive content-type: application/json date: Wed, 15 Sep 2025 02:55:04 GMT keep-alive: timeout=60 transfer-encoding: chunked</pre>

Responses

Code	Description	Links
200	OK	No links

Media type

/

Controls accept header

Example Value Schema

```
{
  "date": "2025-09-15",
  "totalOrders": 0,
  "totalRevenue": 0
}
```

Figure 13. Swagger UI daily order report endpoint.

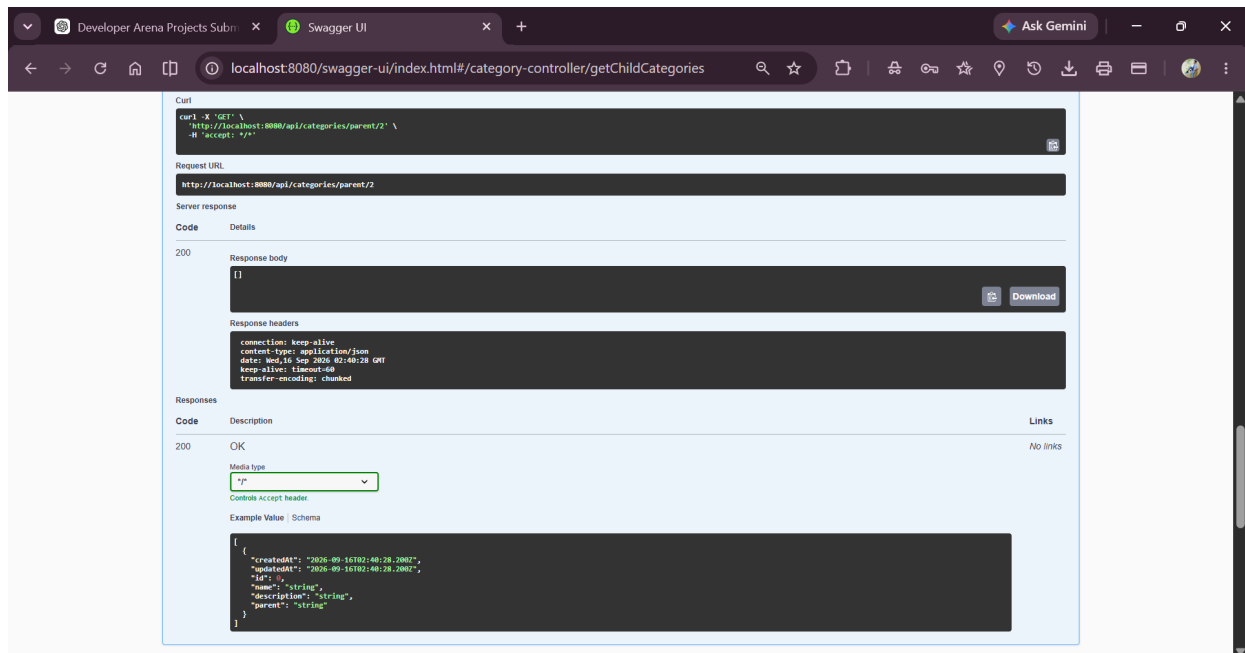


Figure 14. Swagger UI category child-category endpoint.

Developer Arena Projects Subn x Swagger UI x + Ask Gemini

localhost:8080/swagger-ui/index.html#/category-controller/getChildCategories

Curl

```
curl -X 'GET' \
  'http://localhost:8080/api/categories/parent/2' \
  -H 'accept: */*'
```

Request URL

http://localhost:8080/api/categories/parent/2

Server response

Code	Details
500	Error: response status is 500

undocumented

Response body

```
{
  "timestamp": "2026-09-16T08:07:11.557376",
  "status": 500,
  "error": "Internal Server Error",
  "message": "An unexpected error occurred"
}
```

Response headers

```
connection: close
content-type: application/json
date: Wed, 16 Sep 2026 02:37:11 GMT
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	OK	No links

Media type

/

Controls Accept header

Example Value Schema

```
{
  "createdAt": "2026-09-16T02:37:11.606Z",
  "updatedAt": "2026-09-16T02:37:11.606Z",
  "id": 1,
  "name": "string",
  "description": "string",
  "parent": "string"
}
```

Figure 15. Category hierarchy request/response evidence.

Developer Arena Projects Subn x Swagger UI x + Ask Gemini

localhost:8080/swagger-ui/index.html#/category-controller/getChildCategories

Responses

Curl

```
curl -X GET \
  'http://localhost:8080/api/categories/parent/1' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8080/api/categories/parent/1
```

Server response

Code Details

200

Response body

```
{
  "name": "laptops",
  "description": "laptop computers",
  "createdAt": "2026-09-16T08:18:30.733025",
  "id": 1,
  "parent": {
    "name": "Electronics",
    "description": "Electronic devices and accessories",
    "createdAt": "2026-09-16T08:09:27.867758",
    "id": 1,
    "updatedAt": "2026-09-16T08:09:27.867758"
  },
  "updatedAt": "2026-09-16T08:18:30.733025"
}
```

Response headers

```
connection: keep-alive
content-type: application/json
date: Wed, 16 Sep 2026 08:22:43 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	OK	No links

Media type

/

Figure 16. Category hierarchy response evidence.

15. Release and Repository

The source repository is public on GitHub and the submitted release is tagged v1.0.0 and marked as the latest release.

Item	Value
Repository	https://github.com/Ashutosh9-pan/week7-spring-data-jpa
Branch	main
Release	v1.0.0
Release title	Week 7 - Spring Data JPA E-Commerce Backend v1.0.0
README	Repository README contains project overview, setup, architecture, API tables and screenshot references.

Recommended documentation URL for submission: the GitHub repository README or, after uploading this PDF to the repository, the GitHub file page for this PDF.

16. Future Enhancements

- JWT-based authentication and authorization.
- Persistent shopping-cart storage.
- Advanced inventory management.
- Order pagination for large order histories.
- Redis-based distributed caching.
- Docker containerization and CI/CD.
- Production monitoring and metrics.
- Additional PostgreSQL deployment automation.

Conclusion

The Week 7 implementation provides a database-integrated Spring Boot backend with JPA repositories, relational mappings, transactions, locking, migrations, auditing, caching, connection pooling, REST APIs, reporting and automated tests. The v1.0.0 release represents the submitted project state documented in this report.

Prepared for Week 7 - Database Integration with Spring Data JPA